UEMF
الجامعة الأورومتوسطية بفاس
ⵜⴰⵙⴷⴰⵡⵜ ⵜⴰⵎⴰⵡⴰⵙⵜⴰⵏⵜ ⵏ ⴼⴰⵙ
EUROMED UNIVERSITY OF FES

Borne de services UEMF

Technical report: database connections, features and status

Université EuroMed de Fès - DSI | October 2026

Summary

The **Borne de services** is a self-service kiosk. A student or staff member swipes their university card; the kiosk identifies them and lets them check their restaurant balance and purchase history, and reset their Outlook, printer (Active Directory) and Konosys passwords without going to the DSI desk. It is a Python **Flask** web application (*app1.py*) connected to **four databases** (BioStar 1, BioStar 2, Collectivité, Konosys) and **three external services** (Konosys API, Microsoft Graph, Active Directory).

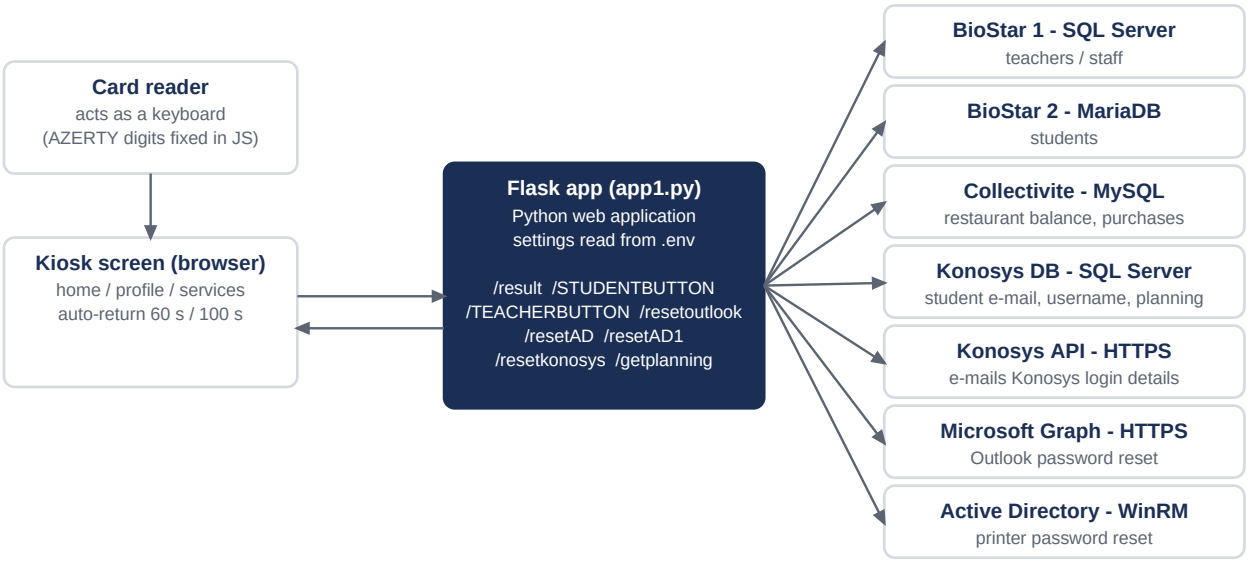
Features and status

Feature	Users	Status	How it works
Card identification	All	**Implemented**	Students in BioStar 2, teachers / staff in BioStar 1.
Restaurant balance	All	**Implemented**	Balance in DH from Collectivité. Shows 0.00 if the database cannot be reached.
Purchase history	All	**Implemented**	Ticket, article, quantity, price, date, time, from Collectivité; newest first.
Help & DSI contact	All	**Implemented**	Static help screen (DSI location, what to do).
Outlook password	All	**Implemented**	New password `uemf@` + 8 digits set via Microsoft Graph, shown on screen, change forced at next sign-in. Needs the user's e-mail.
Konosys password	Students	**Implemented**	Konosys API e-mails the login details to the academic address. Needs only the matricule.
Printer password (AD)	All	**Not functional**	Not working yet; see section 4.
Planning	Students	**Not on screen**	Route /getplanning exists (Konosys procedure) but no button calls it.

How a session works

1. The card reader types the card number into a hidden field (re-focused every second). *kiosk.js* converts AZERTY symbols (&, é, " ...) back to digits; 8 characters minimum.
2. The user chooses **Étudiant** or **Enseignant & personnel** (back to home after 60 s).
3. The app finds the person, then loads balance, e-mail and history. If an optional source fails, the page still opens with a default value; only a failed card lookup shows an error.
4. The services screen shows six tiles. Resets ask for confirmation and block a double tap. Automatic logout after 100 s.

1. Architecture



2. Database connections

All settings come from the *.env* file, loaded by *config.py* (excluded from Git; *.env.example* lists the keys). Exception: Collectivité is still hard-coded in *solde_carte.py*.

Database	Engine / host	Python driver	Used for
BioStar 1 db: biostar	SQL Server 192.168.1.128\BSSERVER	pyodbc {SQL Server}, SQL login, 5 s timeout	Teacher / staff lookup: name, id, e-mail
BioStar 2 db: biostar2_ac	MariaDB 192.168.1.129:3312	pymysql, utf8mb4, 5 s timeout	Student lookup: name, matricule (no e-mail)
Collectivité db: collectivite	MySQL 172.16.80.161:3306	mysql.connector, SSL off, 3 s timeout	Restaurant balance and purchase history
Konosys db: Konosys_UEMF_ Production	SQL Server 172.16.80.120	pyodbc {SQL Server}, SQL login, 5 s timeout	Student e-mail, username, planning procedure

Where each piece of data comes from

Data	Student	Teacher / staff
Name	BioStar 2 (T_USR.NM)	BioStar 1 (TB_USER.sUserName)
Matricule	BioStar 2 custom field "Matricule"	BioStar 1 (TB_USER_CUSTOMINFO.sFieldValue11)
E-mail	Konosys (ERP_Etudiant.MailGEM)	BioStar 1 (TB_USER.sEmail)
AD username	Konosys (ERP_User.Username)	E-mail part before "@"
Balance, history	Collectivité, by matricule	Collectivité, by matricule

Card number: leading zeros are removed. For students, the 4 bytes are reversed (reader gives LSB order, BioStar 2 stores MSB), e.g. 1080807401 becomes 3922684736.

3. Queries and external services

BioStar 1 - teacher by card

```
SELECT TOP (1) a.sUserName, b.sFieldValue11, a.sEmail FROM TB_USER a
JOIN TB_USER_CUSTOMINFO b ON a.nUserIdn = b.nUserIdn
JOIN TB_USER_CARD c ON b.nUserIdn = c.nUserIdn WHERE c.sCardNo = ?
```

BioStar 2 - student by card (custom field 1 = Matricule; newest user if a card was reused)

```
SELECT u.NM, cf.VAL FROM T_USRCRD uc JOIN T_CRD c ON c.CRDUID = uc.CRDUID
JOIN T_USR u ON u.USRUID = uc.USRUID JOIN T_USRCUSFLD cf ON cf.USRUID = uc.USRUID
WHERE TRIM(LEADING '0' FROM c.CRDCSN) = %s AND cf.CUSFLDUID = %s
ORDER BY u.CREATEDT DESC LIMIT 1
```

Konosys - student e-mail and username

```
SELECT Username, MailGEM FROM dbo.ERP_User u
JOIN dbo.ERP_Etudiant e ON u.id_user = e.id_user WHERE MatriculeEtudiant = ?
```

Collectivité - balance and history

```
SELECT nouveau_solde FROM collectivite.compte ce
JOIN collectivite.compt c ON ce.matricule = c.matricule WHERE matricule_Societe = %s
SELECT num_ticket, nom_article, qut_article, famille_art, prix_total_S, prix_total_P,
  date_jour, heure_vente FROM collectivite.ventes v
JOIN collectivite.compt c ON v.num_compte = c.matricule WHERE matricule_Societe = %s
```

Konosys - planning: `EXEC getplanningStudent ?` (matricule)

Service	How the app connects	Settings (.env)
Konosys API	HTTPS POST {"matricule"} with an *X-API-Key* header. 202 = e-mail sent, 404 = unknown matricule, 401/403 = refused.	KONOSYS_API_URL, KONOSYS_API_KEY
Microsoft Graph	OAuth2 client-credentials token, then PATCH /v1.0/users/{e-mail} with a new passwordProfile. 204 = success.	AZURE_CLIENT_ID, AZURE_TENANT_ID, AZURE_CLIENT_SECRET
Active Directory	Python starts PowerShell (Reset-ADPassword.ps1), which runs Invoke-Command on the domain controller MORSRV01N.uemf.ma as UEMF\guichet, then Set-ADAccountPassword.	Hard-coded in the .ps1 script

Student e-mail depends on Konosys. BioStar 2 has no e-mail field, so if the Konosys database cannot be read (e.g. the SQL account has no access to Konosys_UEMF_Production, error 4060), the e-mail and username are empty: the screens show "Non disponible" and the student Outlook and AD resets cannot work.

4. Active Directory (printer password) - not functional yet

Step	What happens	What can block it
1. Username	Students: Konosys *Username*. Teachers: e-mail part before "@".	Empty if Konosys is unreadable. The e-mail prefix may not match the teacher's AD login.
2. PowerShell	ADPasswd.py runs Reset-ADPassword.ps1 (ExecutionPolicy Bypass).	The .ps1 must be deployed next to the application.
3. Remote session	Invoke-Command to MORSRV01N.uemf.ma (WinRM).	The server must resolve the DC name and be trusted by WinRM.
4. AD rights	Set-ADAccountPassword -Reset, then ChangePasswordAtLogon.	UEMF\guichet needs reset rights on student and staff OUs (not verified).
5. Password	`test` + 6 digits, shown on screen.	Uses *random*, not *secrets*; may fail the domain password policy.

To fix: run Reset-ADPassword.ps1 by hand on a test account from the server and read the ERROR line; check `Test-WSMan MORSRV01N.uemf.ma`; confirm the account rights and password policy with the AD admin; use a policy-compliant password generated with *secrets*; read the real AD login for teachers.

5. Known issues and risks

- **Passwords in Git:** plain-text credentials in *solde_carte.py* (Collectivité), *Reset-ADPassword.ps1* (UEMF\guichet) and *api microsoft 365.ps1* (M365 admin, unused). They stay in Git history: the passwords must be changed and moved to *.env*.
- **Shared global state:** the current user is stored in global variables, not in the session. Fine for one kiosk at a time; two screens on the same server would mix users.
- **Silent defaults:** a database failure shows balance 0.00 or an empty history instead of "unavailable".
- **Dead code:** unused routes (/SOLDECARTE, /RESETOUTLOOK, /HISTORY, /PLANNING, /RESETKONOSYS, /SOLDEIMP) pointing to missing templates; old files (app.py, link_data_base.py, empty scripts); no requirements.txt.

6. Next steps

#	Action	Owner
1	Change the passwords exposed in Git and move them to .env.	DSI
2	Confirm the server's Konosys SQL account can read Konosys_UEMF_Production.	DBA / Konosys
3	Make the AD reset work (section 4) and test it on a test account.	DSI + developer
4	Add a Planning tile on the services screen.	Developer
5	Show "unavailable" instead of 0.00; clean dead code; add requirements.txt.	Developer